

Chapter 8. Exactly-Once Semantics

In [Chapter 7](#) we discussed the configuration parameters and the best practices that allow Kafka users to control Kafka's reliability guarantees. We focused on at-least-once delivery—the guarantee that Kafka will not lose messages that it acknowledged as committed. This still leaves open the possibility of duplicate messages.

In simple systems where messages are produced and then consumed by various applications, duplicates are an annoyance that is fairly easy to handle. Most real-world applications contain unique identifiers that consuming applications can use to deduplicate the messages.

Things become more complicated when we look at stream processing applications that aggregate events. When inspecting an application that consumes events, computes an average, and produces the results, it is often impossible for those who check the results to detect that the average is incorrect because an event was processed twice while computing the average. In these cases, it is important to provide a stronger guarantee—exactly-once processing semantics.

In this chapter, we will discuss how to use Kafka with exactly-once semantics, the recommended use cases, and the limitations. As we did with at-least-once guarantees, we will dive a bit deeper and provide some insight and intuition into how this guarantee is implemented. These details can be skipped when you first read the chapter but will be useful to understand before using the feature—it will help clarify the meaning of the different configurations and APIs and how best to use them.

Exactly-once semantics in Kafka is a combination of two key features: idempotent producers, which help avoid duplicates caused by producer retries, and transactional semantics, which guarantee exactly-once processing in stream processing applications. We will discuss both, starting with the simpler and more generally useful idempotent producer.

Idempotent Producer

A service is called idempotent if performing the same operation multiple times has the same result as performing it a single time. In databases it is usually demonstrated as the difference between `UPDATE t SET x=x+1`

where $y=5$ and `UPDATE t SET x=18 where y=5`. The first example is not idempotent; if we call it three times, we'll end up with a very different result than if we were to call it once. The second example is idempotent—no matter how many times we run this statement, `x` will be equal to 18.

How is this related to a Kafka producer? If we configure a producer to have at-least-once semantics rather than idempotent semantics, it means that in cases of uncertainty, the producer will retry sending the message so it will arrive at least once. These retries could lead to duplicates.

The classic case is when a partition leader received a record from the producer, replicated it successfully to the followers, and then the broker on which the leader resides crashed before it could send a response to the producer. The producer, after a certain time without a response, will re-send the message. The message will arrive at the new leader, who already has a copy of the message from the previous attempt—resulting in a duplicate.

In some applications duplicates don't matter much, but in others they can lead to inventory miscounts, bad financial statements, or sending someone two umbrellas instead of the one they ordered.

Kafka's idempotent producer solves this problem by automatically detecting and resolving such duplicates.

How Does the Idempotent Producer Work?

When we enable the idempotent producer, each message will include a unique identified producer ID (PID) and a sequence number. These, together with the target topic and partition, uniquely identify each message. Brokers use these unique identifiers to track the last five messages produced to every partition on the broker. To limit the number of previous sequence numbers that have to be tracked for each partition, we also require that the producers will use `max.inflight.requests=5` or lower (the default is 5).

When a broker receives a message that it already accepted before, it will reject the duplicate with an appropriate error. This error is logged by the producer and is reflected in its metrics but does not cause any exception and should not cause any alarm. On the producer client, it will be added to the `record-error-rate` metric. On the broker, it will be part of the `ErrorsPerSec` metric of the `RequestMetrics` type, which includes a separate count for each type of error.

What if a broker receives a sequence number that is unexpectedly high? The broker expects message number 2 to be followed by message number

3; what happens if the broker receives message number 27 instead? In such cases the broker will respond with an “out of order sequence” error, but if we use an idempotent producer without using transactions, this error can be ignored.

WARNING

While the producer will continue normally after encountering an “out of order sequence number” exception, this error typically indicates that messages were lost between the producer and the broker—if the broker received message number 2 followed by message number 27, something must have happened to messages 3 to 26. When encountering such an error in the logs, it is worth revisiting the producer and topic configuration and making sure the producer is configured with recommended values for high reliability and to check whether unclean leader election has occurred.

As is always the case with distributed systems, it is interesting to consider the behavior of an idempotent producer under failure conditions. Consider two cases: producer restart and broker failure.

Producer restart

When a producer fails, usually a new producer will be created to replace it—whether manually by a human rebooting a machine, or using a more sophisticated framework like Kubernetes that provides automated failure recovery. The key point is that when the producer starts, if the idempotent producer is enabled, the producer will initialize and reach out to a Kafka broker to generate a producer ID. Each initialization of a producer will result in a completely new ID (assuming that we did not enable transactions). This means that if a producer fails and the producer that replaces it sends a message that was previously sent by the old producer, the broker will not detect the duplicates—the two messages will have different producer IDs and different sequence numbers and will be considered as two different messages. Note that the same is true if the old producer froze and then came back to life after its replacement started—the original producer is not recognized as a zombie, because we have two totally different producers with different IDs.

Broker failure

When a broker fails, the controller elects new leaders for the partitions that had leaders on the failed broker. Say that we have a producer that produced messages to topic A, partition 0, which had its lead replica on broker 5 and a follower replica on broker 3. After broker 5 fails, broker 3 becomes the new leader. The producer will discover that the new leader

is broker 3 via the metadata protocol and start producing to it. But how will broker 3 know which sequences were already produced in order to reject duplicates?

The leader keeps updating its in-memory producer state with the five last sequence IDs every time a new message is produced. Follower replicas update their own in-memory buffers every time they replicate new messages from the leader. This means that when a follower becomes a leader, it already has the latest sequence numbers in memory, and validation of newly produced messages can continue without any issues or delays.

But what happens when the old leader comes back? After a restart, the old in-memory producer state will no longer be in memory. To assist in recovery, brokers take a snapshot of the producer state to a file when they shut down or every time a segment is created. When the broker starts, it reads the latest state from a file. The newly restarted broker then keeps updating the producer state as it catches up by replicating from the current leader, and it has the most current sequence IDs in memory when it is ready to become a leader again.

What if a broker crashed and the last snapshot is not updated? Producer ID and sequence ID are also part of the message format that is written to Kafka's logs. During crash recovery, the producer state will be recovered by reading the older snapshot and also messages from the latest segment of each partition. A new snapshot will be stored as soon as the recovery process completes.

An interesting question is what happens if there are no messages? Imagine that a certain topic has two hours of retention time, but no new messages arrived in the last two hours—there will be no messages to use to recover the state if a broker crashed. Luckily, no messages also means no duplicates. We will start accepting messages immediately (while logging a warning about the lack of state), and create the producer state from the new messages that arrive.

Limitations of the Idempotent Producer

Kafka's idempotent producer only prevents duplicates in case of retries that are caused by the producer's internal logic. Calling `producer.send()` twice with the same message will create a duplicate, and the idempotent producer won't prevent it. This is because the producer has no way of knowing that the two records that were sent are in fact the same record. It is always a good idea to use the built-in retry mechanism of the producer rather than catching producer exceptions and retrying from the application itself; the idempotent producer makes

this pattern even more appealing—it is the easiest way to avoid duplicates when retrying.

It is also rather common to have applications that have multiple instances or even one instance with multiple producers. If two of these producers attempt to send identical messages, the idempotent producer will not detect the duplication. This scenario is fairly common in applications that get data from a source—a directory with files, for instance—and produce it to Kafka. If the application happened to have two instances reading the same file and producing records to Kafka, we will get multiple copies of the records in that file.

TIP

The idempotent producer will only prevent duplicates caused by the retry mechanism of the producer itself, whether the retry is caused by producer, network, or broker errors. But nothing else.

How Do I Use the Kafka Idempotent Producer?

This is the easy part. Add `enable.idempotence=true` to the producer configuration. If the producer is already configured with `acks=all`, there will be no difference in performance. By enabling idempotent producer, the following things will change:

- To retrieve a producer ID, the producer will make one extra API call when starting up.
- Each record batch sent will include the producer ID and the sequence ID for the first message in the batch (sequence IDs for each message in the batch are derived from the sequence ID of the first message plus a delta). These new fields add 96 bits to each record batch (producer ID is a long, and sequence is an integer), which is barely any overhead for most workloads.
- Brokers will validate the sequence numbers from any single producer instance and guarantee the lack of duplicate messages.
- The order of messages produced to each partition will be guaranteed, through all failure scenarios, even if `max.in.flight.requests.per.connection` is set to more than 1 (5 is the default and also the highest value supported by the idempotent producer).

NOTE

Idempotent producer logic and error handling improved significantly in version 2.5 (both on the producer side and the broker side) as a result of KIP-360. Prior to release 2.5, the producer state was not always maintained for long enough, which resulted in fatal UNKNOWN_PRODUCER_ID errors in various scenarios (partition reassignment had a known edge case where the new replica became the leader before any writes happened from a specific producer, meaning that the new leader had no state for that partition). In addition, previous versions attempted to rewrite the sequence IDs in some error scenarios, which could lead to duplicates. In newer versions, if we encounter a fatal error for a record batch, this batch and all the batches that are in flight will be rejected. The user who writes the application can handle the exception and decide whether to skip those records or retry and risk duplicates and reordering.

Transactions

As we mentioned in the introduction to this chapter, transactions were added to Kafka to guarantee the correctness of applications developed using Kafka Streams. In order for a stream processing application to generate correct results, each input record must be processed exactly one time, and its processing result will be reflected exactly one time, even in case of failure. Transactions in Apache Kafka allow stream processing applications to generate accurate results. This, in turn, enables developers to use stream processing applications in use cases where accuracy is a key requirement.

It is important to keep in mind that transactions in Kafka were developed specifically for stream processing applications. And therefore they were built to work with the “consume-process-produce” pattern that forms the basis of stream processing applications. Use of transactions can guarantee exactly-once semantics in this context—the processing of each input record will be considered complete after the application’s internal state has been updated and the results were successfully produced to output topics. In [“What Problems Aren’t Solved by Transactions?”](#), we’ll explore a few scenarios where Kafka’s exactly-once guarantees will not apply.

NOTE

Transactions is the name of the underlying mechanism. Exactly-once semantics or exactly-once guarantees is the behavior of a stream processing application. Kafka Streams uses transactions to implement its exactly-once guarantees. Other stream processing frameworks, such as Spark Streaming or Flink, use different mechanisms to provide their users with exactly-once semantics.

Transactions Use Cases

Transactions are useful for any stream processing application where accuracy is important, and especially where stream processing includes aggregation and/or joins. If the stream processing application only performs single record transformation and filtering, there is no internal state to update, and even if duplicates were introduced in the process, it is fairly straightforward to filter them out of the output stream. When the stream processing application aggregates several records into one, it is much more difficult to check whether a result record is wrong because some input records were counted more than once; it is impossible to correct the result without reprocessing the input.

Financial applications are typical examples of complex stream processing applications where exactly-once capabilities are used to guarantee accurate aggregation. However, because it is rather trivial to configure any Kafka Streams application to provide exactly-once guarantees, we've seen it enabled in more mundane use cases, including, for instance, chatbots.

What Problems Do Transactions Solve?

Consider a simple stream processing application: it reads events from a source topic, maybe processes them, and writes results to another topic. We want to be sure that for each message we process, the results are written exactly once. What can possibly go wrong?

It turns out that quite a few things could go wrong. Let's look at two scenarios.

Reprocessing caused by application crashes

After consuming a message from the source cluster and processing it, the application has to do two things: produce the result to the output topic, and commit the offset of the message that we consumed. Suppose that these two separate actions happen in this order. What happens if the application crashes after the output was produced but before the offset of the input was committed?

In [Chapter 4](#), we discussed what happens when a consumer crashes. After a few seconds, the lack of heartbeat will trigger a rebalance, and the partitions the consumer was consuming from will be reassigned to a different consumer. That consumer will begin consuming records from those partitions, starting at the last committed offset. This means that all the records that were processed by the application between the last commit-

ted offset and the crash will be processed again, and the results will be written to the output topic again—resulting in duplicates.

Reprocessing caused by zombie applications

What happens if our application just consumed a batch of records from Kafka and then froze or lost connectivity to Kafka before doing anything else with this batch of records?

Just like in the previous scenario, after several heartbeats are missed, the application will be assumed dead and its partitions reassigned to another consumer in the consumer group. That consumer will reread that batch of records, process it, produce the results to an output topic, and continue on.

Meanwhile, the first instance of the application—the one that froze—may resume its activity: process the batch of records it recently consumed, and produce the results to the output topic. It can do all that before it polls Kafka for records or sends a heartbeat and discovers that it is supposed to be dead and another instance now owns those partitions.

A consumer that is dead but doesn't know it is called a zombie. In this scenario, we can see that without additional guarantees, zombies can produce data to the output topic and cause duplicate results.

How Do Transactions Guarantee Exactly-Once?

Take our simple stream processing application. It reads data from one topic, processes it, and writes the result to another topic. Exactly-once processing means that consuming, processing, and producing are done *atomically*. Either the offset of the original message is committed and the result is successfully produced or neither of these things happen. We need to make sure that partial results—where the offset is committed but the result isn't produced, or vice versa—can't happen.

To support this behavior, Kafka transactions introduce the idea of *atomic multipartition writes*. The idea is that committing offsets and producing results both involve writing messages to partitions. However, the results are written to an output topic, and offsets are written to the `_consumer_offsets` topic. If we can open a transaction, write both messages, and commit if both were written successfully—or abort to retry if they were not—we will get the exactly-once semantics that we are after.

[Figure 8-1](#) illustrates a simple stream processing application, performing an atomic multipartition write to two partitions while also committing offsets for the event it consumed.

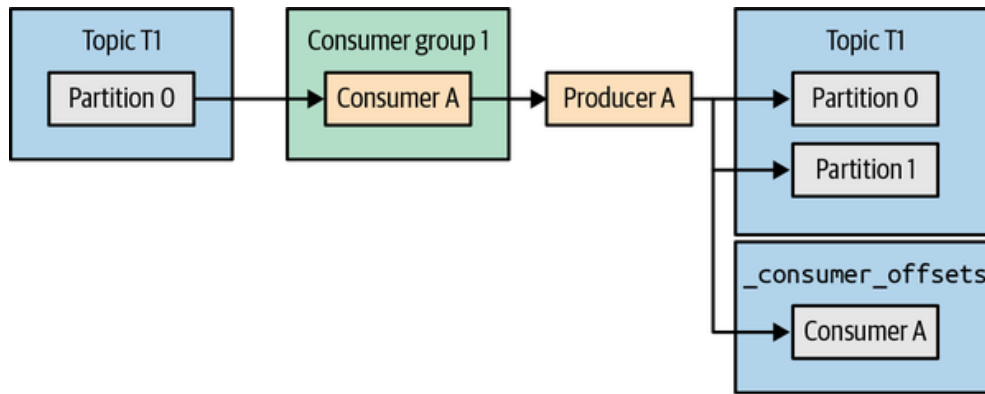


Figure 8-1. Transactional producer with atomic multipartition write

To use transactions and perform atomic multipartition writes, we use a *transactional producer*. A transactional producer is simply a Kafka producer that is configured with a `transactional.id` and has been initialized using `initTransactions()`. Unlike `producer.id`, which is generated automatically by Kafka brokers, `transactional.id` is part of the producer configuration and is expected to persist between restarts. In fact, the main role of the `transactional.id` is to identify the same producer across restarts. Kafka brokers maintain `transactional.id` to `producer.id` mapping, so if `initTransactions()` is called again with an existing `transactional.id`, the producer will also be assigned the same `producer.id` instead of a new random number.

Preventing zombie instances of the application from creating duplicates requires a mechanism for *zombie fencing*, or preventing zombie instances of the application from writing results to the output stream. The usual way of fencing zombies—using an epoch—is used here. Kafka increments the epoch number associated with a `transactional.id` when `initTransaction()` is invoked to initialize a transactional producer. Send, commit, and abort requests from producers with the same `transactional.id` but lower epochs will be rejected with the `FencedProducer` error. The older producer will not be able to write to the output stream and will be forced to `close()`, preventing the zombie from introducing duplicate records. In Apache Kafka 2.5 and later, there is also an option to add consumer group metadata to the transaction metadata. This metadata will also be used for fencing, which will allow producers with different transactional IDs to write to the same partitions while still fencing against zombie instances.

Transactions are a producer feature for the most part—we create a transactional producer, begin the transaction, write records to multiple partitions, produce offsets in order to mark records as already processed, and

commit or abort the transaction. We do all this from the producer. However, this isn't quite enough—records written transactionally, even ones that are part of transactions that were eventually aborted, are written to partitions just like any other records. Consumers need to be configured with the right isolation guarantees, otherwise we won't have the exactly-once guarantees we expected.

We control the consumption of messages that were written transactionally by setting the `isolation.level` configuration. If set to `read_committed`, calling `consumer.poll()` after subscribing to a set of topics will return messages that were either part of a successfully committed transaction or that were written nontransactionally; it will not return messages that were part of an aborted transaction or a transaction that is still open. The default `isolation.level` value, `read_uncommitted`, will return all records, including those that belong to open or aborted transactions. Configuring `read_committed` mode does not guarantee that the application will get all messages that are part of a specific transaction. It is possible to subscribe to only a subset of topics that were part of the transaction and therefore get a subset of the messages. In addition, the application can't know when transactions begin or end, or which messages are part of which transaction.

[Figure 8-2](#) shows which records are visible to a consumer in `read_committed` mode compared to a consumer with the default `read_uncommitted` mode.

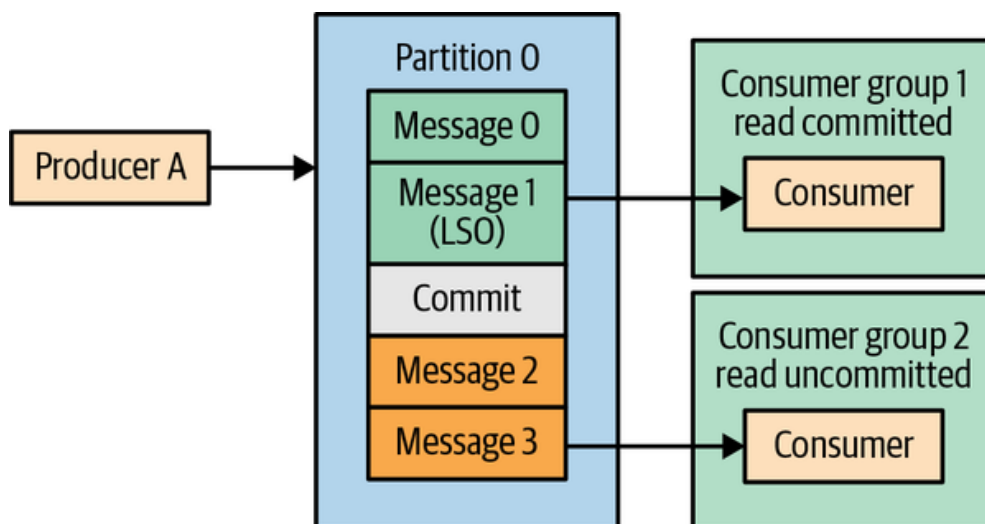


Figure 8-2. Consumers in `read_committed` mode will lag behind consumers with default configuration

To guarantee that messages will be read in order, `read_committed` mode will not return messages that were produced after the point when the first still-open transaction began (known as the Last Stable Offset, or LSO). Those messages will be withheld until that transaction is committed or aborted by the producer, or until they reach `transaction.timeout.ms` (default of 15 minutes) and are aborted by

the broker. Holding a transaction open for a long duration will introduce higher end-to-end latency by delaying consumers.

Our simple stream processing job will have exactly-once guarantees on its output even if the input was written nontransactionally. The atomic multipartition produce guarantees that if the output records were committed to the output topic, the offset of the input records was also committed for that consumer, and as a result the input records will not be processed again.

What Problems Aren't Solved by Transactions?

As explained earlier, transactions were added to Kafka to provide multipartition atomic writes (but not reads) and to fence zombie producers in stream processing applications. As a result, they provide exactly-once guarantees when used within chains of consume-process-produce stream processing tasks. In other contexts, transactions will either straight-out not work or will require additional effort in order to achieve the guarantees we want.

The two main mistakes are assuming that exactly-once guarantees apply on actions other than producing to Kafka, and that consumers always read entire transactions and have information about transaction boundaries.

The following are a few scenarios in which Kafka transactions won't help achieve exactly-once guarantees.

Side effects while stream processing

Let's say that the record processing step in our stream processing app includes sending email to users. Enabling exactly-once semantics in our app will not guarantee that the email will only be sent once. The guarantee only applies to records written to Kafka. Using sequence numbers to deduplicate records or using markers to abort or to cancel a transaction works within Kafka, but it will not un-send an email. The same is true for any action with external effects that is performed within the stream processing app: calling a REST API, writing to a file, etc.

Reading from a Kafka topic and writing to a database

In this case, the application is writing to an external database rather than to Kafka. In this scenario, there is no producer involved—records are written to the database using a database driver (likely JDBC) and offsets are committed to Kafka within the consumer. There is no mechanism that allows writing results to an external database and committing offsets to

Kafka within a single transaction. Instead, we could manage offsets in the database (as explained in [Chapter 4](#)) and commit both data and offsets to the database in a single transaction—this would rely on the database’s transactional guarantees rather than Kafka’s.

NOTE

Microservices often need to update the database *and* publish a message to Kafka within a single atomic transaction, so either both will happen or neither will. As we’ve just explained in the last two examples, Kafka transactions will not do this.

A common solution to this common problem is known as the *outbox pattern*. The microservice only publishes the message to a Kafka topic (the “outbox”), and a separate message relay service reads the event from Kafka and updates the database. Because, as we’ve just seen, Kafka won’t guarantee an exactly-once update to the database, it is important to make sure the update is idempotent.

Using this pattern guarantees that the message will eventually make it to Kafka, the topic consumers, and the database—or to none of those.

The inverse pattern—where a database table serves as the outbox and a relay service makes sure updates to the table will also arrive to Kafka as messages—is also used. This pattern is preferred when built-in RDBMS constraints, such as uniqueness and foreign keys, are useful. The Debezium project published an [in-depth blog post on the outbox pattern](#) with detailed examples.

Reading data from a database, writing to Kafka, and from there writing to another database

It is very tempting to believe that we can build an app that will read data from a database, identify database transactions, write the records to Kafka, and from there write records to another database, still maintaining the original transactions from the source database.

Unfortunately, Kafka transactions don’t have the necessary functionality to support these kinds of end-to-end guarantees. In addition to the problem with committing both records and offsets within the same transaction, there is another difficulty: `read_committed` guarantees in Kafka consumers are too weak to preserve database transactions. Yes, a consumer will not see records that were not committed. But it is not guaranteed to have seen all the records that were committed within the transaction because it could be lagging on some topics; it has no information to identify transaction boundaries, so it can’t know when a transaction began and ended, and whether it has seen some, none, or all of its records.

Copying data from one Kafka cluster to another

This one is more subtle—it is possible to support exactly-once guarantees when copying data from one Kafka cluster to another. There is a description of how this is done in the Kafka improvement proposal for adding [exactly-once capabilities in MirrorMaker 2.0](#). At the time of this writing, the proposal is still in draft, but the algorithm is clearly described. This proposal includes the guarantee that each record in the source cluster will be copied to the destination cluster exactly once.

However, this does not guarantee that transactions will be atomic. If an app produces several records and offsets transactionally, and then MirrorMaker 2.0 copies them to another Kafka cluster, the transactional properties and guarantees will be lost during the copy process. They are lost for the same reason when copying data from Kafka to a relational database: the consumer reading data from Kafka can't know or guarantee that it is getting all the events in a transaction. For example, it can replicate part of a transaction if it is only subscribed to a subset of the topics.

Publish/subscribe pattern

Here's a slightly more subtle case. We've discussed exactly-once in the context of the consume-process-produce pattern, but the publish/subscribe pattern is a very common use case. Using transactions in a publish/subscribe use case provides some guarantees: consumers configured with `read_committed` mode will not see records that were published as part of a transaction that was aborted. But those guarantees fall short of exactly-once. Consumers may process a message more than once, depending on their own offset commit logic.

The guarantees Kafka provides in this case are similar to those provided by JMS transactions but depend on consumers in `read_committed` mode to guarantee that uncommitted transactions will remain invisible. JMS brokers withhold uncommitted transactions from all consumers.

WARNING

An important pattern to avoid is publishing a message and then waiting for another application to respond before committing the transaction. The other application will not receive the message until after the transaction was committed, resulting in a deadlock.

How Do I Use Transactions?

Transactions are a broker feature and part of the Kafka protocol, so there are multiple clients that support transactions.

The most common and most recommended way to use transactions is to enable exactly-once guarantees in Kafka Streams. This way, we will not use transactions directly at all, but rather Kafka Streams will use them for us behind the scenes to provide the guarantees we need. Transactions were designed with this use case in mind, so using them via Kafka Streams is the easiest and most likely to work as expected.

To enable exactly-once guarantees for a Kafka Streams application, we simply set the `processing.guarantee` configuration to either `exactly_once` or `exactly_once_beta`. That's it.

NOTE

`exactly_once_beta` is a slightly different method of handling application instances that crash or hang with in-flight transactions. This was introduced in release 2.5 to Kafka brokers, and in release 2.6 to Kafka Streams. The main benefit of this method is the ability to handle many partitions with a single transactional producer and therefore create more scalable Kafka Streams applications. There is more information about the changes in the [Kafka improvement proposal where they were first discussed](#).

But what if we want exactly-once guarantees without using Kafka Streams? In this case we will use transactional APIs directly. Here's a snippet showing how this will work. There is a full example in the Apache Kafka GitHub, which includes a [demo driver](#) and a [simple exactly-once processor](#) that runs in separate threads:

```
Properties producerProps = new Properties();
producerProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
producerProps.put(ProducerConfig.CLIENT_ID_CONFIG, "DemoProducer");
producerProps.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, transactionalId);

producer = new KafkaProducer<>(producerProps);

Properties consumerProps = new Properties();
consumerProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);
consumerProps.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false"); ❷
consumerProps.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");

consumer = new KafkaConsumer<>(consumerProps);
```

```

producer.initTransactions(); ❹

consumer.subscribe(Collections.singleton(inputTopic)); ❺

while (true) {
    try {
        ConsumerRecords<Integer, String> records =
            consumer.poll(Duration.ofMillis(200));
        if (records.count() > 0) {
            producer.beginTransaction(); ❻
            for (ConsumerRecord<Integer, String> record : records) {
                ProducerRecord<Integer, String> customizedRecord = transform(record);
                producer.send(customizedRecord);
            }
            Map<TopicPartition, OffsetAndMetadata> offsets = consumerOffsets();
            producer.sendOffsetsToTransaction(offsets, consumer.groupMetadata());
            producer.commitTransaction(); ❼
        }
    } catch (ProducerFencedException|InvalidProducerEpochException e) { ❽
        throw new KafkaException(String.format(
            "The transactional.id %s is used by another process", transactionalId));
    } catch (KafkaException e) {
        producer.abortTransaction(); ❾
        resetToLastCommittedPositions(consumer);
    }
}

```

- ❶ Configuring a producer with `transactional.id` makes it a transactional producer capable of producing atomic multipartition writes. The transactional ID must be unique and long-lived. Essentially it defines an instance of the application.
- ❷ Consumers that are part of the transactions don't commit their own offsets—the producer writes offsets as part of the transaction. So offset commit should be disabled.
- ❸ In this example, the consumer reads from an input topic. We will assume that the records in the input topic were also written by a transactional producer (just for fun—there is no such requirement for the input). To read transactions cleanly (i.e., ignore in-flight and aborted transactions), we will set the consumer isolation level to `read_committed`. Note that the consumer will still read nontransactional writes, in addition to reading committed transactions.
- ❹ The first thing a transactional producer must do is initialize. This registers the transactional ID, bumps up the epoch to guarantee that other producers with the same ID will be considered zombies, and aborts older in-flight transactions from the same transactional ID.

- ⑤ Here we are using the `subscribe` consumer API, which means that partitions assigned to this instance of the application can change at any point as a result of rebalance. Prior to release 2.5, which introduced API changes from KIP-447, this was much more challenging. Transactional producers had to be statically assigned a set of partitions, because the transaction fencing mechanism relied on the same transactional ID being used for the same partitions (there was no zombie fencing protection if the transactional ID changed). KIP-447 added new APIs, used in this example, that attach consumer-group information to the transaction, and this information is used for fencing. When using this method, it also makes sense to commit transactions whenever the related partitions are revoked.
- ⑥ We consumed records, and now we want to process them and produce results. This method guarantees that everything that is produced from the time it was called, until the transaction is either committed or aborted, is part of a single atomic transaction.
- ⑦ This is where we process the records—all our business logic goes here.
- ⑧ As we explained earlier in the chapter, it is important to commit the offsets as part of the transaction. This guarantees that if we fail to produce results, we won't commit the offsets for records that were not, in fact, processed. This method commits offsets as part of the transaction. Note that it is important not to commit offsets in any other way—disable offset auto-commit, and don't call any of the consumer commit APIs. Committing offsets by any other method does not provide transactional guarantees.
- ⑨ We produced everything we needed, we committed offsets as part of the transaction, and it is time to commit the transaction and seal the deal. Once this method returns successfully, the entire transaction has made it through, and we can continue to read and process the next batch of events.
- ⑩ If we got this exception, it means we are the zombie. Somehow our application froze or disconnected, and there is a newer instance of the app with our transactional ID running. Most likely the transaction we started has already been aborted and someone else is processing those records. Nothing to do but die gracefully.
- ⑪ If we got an error while writing a transaction, we can abort the transaction, set the consumer position back, and try again.

Transactional IDs and Fencing

Choosing the transactional ID for producers is important and a bit more challenging than it seems. Assigning the transactional ID incorrectly can lead to either application errors or loss of exactly-once guarantees. The key requirements are that the transactional ID will be consistent for the same instance of the application between restarts and is different for different instances of the application, otherwise the brokers will not be able to fence off zombie instances.

Until release 2.5, the only way to guarantee fencing was to statically map the transactional ID to partitions. This guaranteed that each partition will always be consumed with the same transactional ID. If a producer with transactional ID A processed messages from topic T and lost connectivity, and the new producer that replaces it has transactional ID B, and later producer A comes back as a zombie, zombie A will not be fenced because the ID doesn't match that of the new producer B. We want producer A to always be replaced by producer A, and the new producer A will have a higher epoch number and zombie A will be properly fenced away. In those releases, the previous example would be incorrect—transactional IDs are assigned randomly to threads without making sure the same transactional ID is always used to write to the same partition.

In Apache Kafka 2.5, KIP-447 introduced a second method of fencing based on consumer group metadata for fencing in addition to transactional IDs. We use the producer offset commit method and pass as an argument the consumer group metadata rather than just the consumer group ID.

Let's say that we have topic T1 with two partitions, t-0 and t-1. Each is consumed by a separate consumer in the same group; each consumer passes records to a matching transactional producer—one with transactional ID A and the other with transactional ID B; and they are writing output to topic T2 partitions 0 and 1, respectively. [Figure 8-3](#) illustrates this scenario.

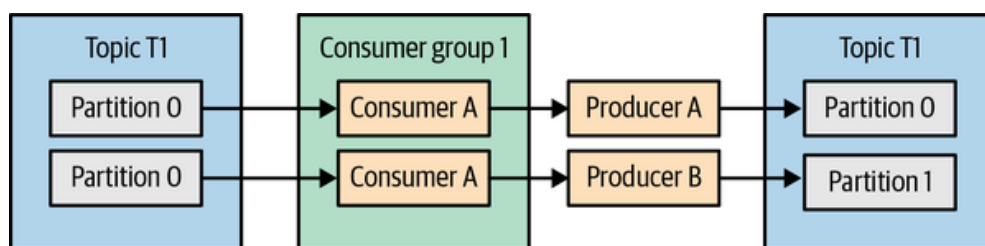


Figure 8-3. Transactional record processor

As illustrated in [Figure 8-4](#), if the application instance with consumer A and producer A becomes a zombie, consumer B will start processing

records from both partitions. If we want to guarantee that no zombies write to partition 0, consumer B can't just start reading from partition 0 and writing to partition 0 with transactional ID B. Instead the application will need to instantiate a new producer, with transactional ID A, to safely write to partition 0 and fence the old transactional ID A. This is wasteful. Instead, we include the consumer group information in the transactions. Transactions from producer B will show that they are from a newer generation of the consumer group, and therefore they will go through, while transactions from the now-zombie producer A will show an old generation of the consumer group and will be fenced.

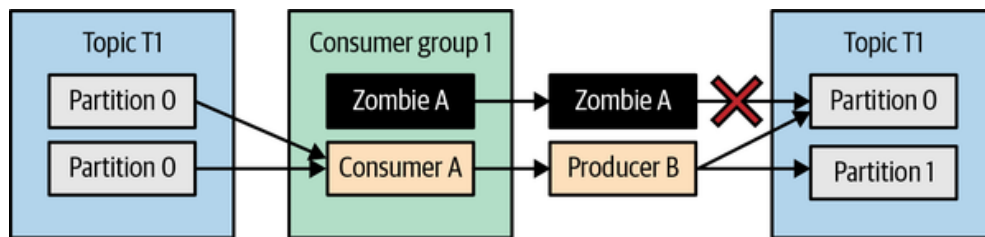


Figure 8-4. Transactional record processor after a rebalance

How Transactions Work

We can use transactions by calling the APIs without understanding how they work. But having some mental model of what is going on under the hood will help us troubleshoot applications that do not behave as expected.

The basic algorithm for transactions in Kafka was inspired by Chandy-Lamport snapshots, in which “marker” control messages are sent into communication channels, and consistent state is determined based on the arrival of the marker. Kafka transactions use marker messages to indicate that transactions are committed or aborted across multiple partitions—when the producer decides to commit a transaction, it sends a “commit” message to the transaction coordinator, which then writes commit markers to all partitions involved in a transaction. But what happens if the producer crashes after only writing commit messages to a subset of the partitions? Kafka transactions solve this by using two-phase commit and a transaction log. At a high level, the algorithm will:

1. Log the existence of an ongoing transaction, including the partitions involved
2. Log the intent to commit or abort—once this is logged, we are doomed to commit or abort eventually
3. Write all the transaction markers to all the partitions
4. Log the completion of the transaction

To implement this basic algorithm, Kafka needs a transaction log. We use an internal topic called `__transaction_state`.

Let's see how this algorithm works in practice by going through the inner workings of the transactional API calls we've used in the preceding code snippet.

Before we begin the first transaction, producers need to register as transactional by calling `initTransaction()`. This request is sent to a broker that will be the *transaction coordinator* for this transactional producer. Each broker is the transactional coordinator for a subset of the producers, just like each broker is the consumer group coordinator for a subset of the consumer groups. The transaction coordinator for each transactional ID is the leader of the partition of the transaction log the transactional ID is mapped to.

The `initTransaction()` API registers a new transactional ID with the coordinator, or increments the epoch of an existing transactional ID in order to fence off previous producers that may have become zombies. When the epoch is incremented, pending transactions will be aborted.

The next step for the producer is to call `beginTransaction()`. This API call isn't part of the protocol—it simply tells the producer that there is now a transaction in progress. The transaction coordinator on the broker side is still unaware that the transaction began. However, once the producer starts sending records, each time the producer detects that it is sending records to a new partition, it will also send `AddPartitionsToTxnRequest` to the broker informing it that there is a transaction in progress for this producer, and that additional partitions are part of the transaction. This information will be recorded in the transaction log.

When we are done producing results and are ready to commit, we start by committing offsets for the records we've processed in this transaction. Committing offsets can be done at any time but must be done before the transaction is committed. Calling `sendOffsetsToTransaction()` will send a request to the transaction coordinator that includes the offsets and also the consumer group ID. The transaction coordinator will use the consumer group ID to find the group coordinator and commit the offsets as a consumer group normally would.

Now it is time to commit—or abort. Calling `commitTransaction()` or `abortTransaction()` will send an `EndTransactionRequest` to the transaction coordinator. The transaction coordinator will log the commit or abort intention to the transaction log. Once this step is successful, it is the transaction coordinator's responsibility to complete the commit (or

abort) process. It writes a commit marker to all the partitions involved in the transaction, then writes to the transaction log that the commit completed successfully. Note that if the transaction coordinator shuts down or crashes after logging the intention to commit and before completing the process, a new transaction coordinator will be elected, pick up the intent to commit from the transaction log, and complete the process.

If a transaction is not committed or aborted within `transaction.timeout.ms`, the transaction coordinator will abort it automatically.

WARNING

Each broker that receives records from transactional or idempotent producers will store the producer/transactional IDs in memory, together with related state for each of the last five batches sent by the producer: sequence numbers, offsets, and such. This state is stored for `transactional.id.expiration.ms` milliseconds after the producer stopped being active (seven days by default). This allows the producer to resume activity without running into `UNKNOWN_PRODUCER_ID` errors. It is possible to cause something similar to a memory leak in the broker by creating new idempotent producers or new transactional IDs at a very high rate but never reusing them. Three new idempotent producers per second, accumulated over the course of a week, will result in 1.8 million producer state entries with a total of 9 million batch metadata stored, using around 5 GB RAM. This can cause out-of-memory or severe garbage collection issues on the broker. We recommend architecting the application to initialize a few long-lived producers when the application starts up, and then reuse them for the lifetime of the application. If this isn't possible (Function as a Service makes this difficult), we recommend lowering `transactional.id.expiration.ms` so the IDs will expire faster, and therefore old state that will never be reused won't take up a significant part of the broker memory.

Performance of Transactions

Transactions add moderate overhead to the producer. The request to register transactional ID occurs once in the producer lifecycle. Additional calls to register partitions as part of a transaction happen at most one per partition for each transaction, then each transaction sends a commit request, which causes an extra commit marker to be written on each partition. The transactional initialization and transaction commit requests are synchronous, so no data will be sent until they complete successfully, fail, or time out, which further increases the overhead.

Note that the overhead of transactions on the producer is independent of the number of messages in a transaction. So a larger number of messages

per transaction will both reduce the relative overhead and reduce the number of synchronous stops, resulting in higher throughput overall.

On the consumer side, there is some overhead involved in reading commit markers. The key impact that transactions have on consumer performance is introduced by the fact that consumers in `read_committed` mode will not return records that are part of an open transaction. Long intervals between transaction commits mean that the consumer will need to wait longer before returning messages, and as a result, end-to-end latency will increase.

Note, however, that the consumer does not need to buffer messages that belong to open transactions. The broker will not return those in response to fetch requests from the consumer. Since there is no extra work for the consumer when reading transactions, there is no decrease in throughput either.

Summary

Exactly-once semantics in Kafka is the opposite of chess: it is challenging to understand but easy to use.

This chapter covered the two key mechanisms that provide exactly-once guarantees in Kafka: idempotent producer, which avoids duplicates that are caused by the retry mechanism, and transactions, which form the basis of exactly-once semantics in Kafka Streams.

Both can be enabled in a single configuration and allow us to use Kafka for applications that require fewer duplicates and stronger correctness guarantees.

We discussed in depth specific scenarios and use cases to show the expected behavior, and even looked at some of the implementation details. Those details are important when troubleshooting applications or when using transactional APIs directly.

By understanding what Kafka's exactly-once semantics guarantee in which use case, we can design applications that will use exactly-once when necessary. Application behavior should not be surprising, and the information in this chapter will help us avoid surprises.